Project Dev ops

1:พร็อมให้Ai ตัวไหนก็ได้ตามข้อความด้านล่างนี้↓
มีโปรเจ็คdevops อาจารย์ให้ติดตั้งlocal ai ลงdocker ผ่านollama ด้วยสเปคเครื่อง [ใส่สเปคเครื่องตรงนี้ *ระบุ cpu ,ram ,gpu(ถ้ามี)] โมเดลตัวไหนเหมาะบ้างสัก2ตัว สอนทุกขั้นตอนตั้งแต่ติดตั้งจนถึงใช้งานในvscode



*มันก็จะวิเคราะห์สเปค แล้วหาตัวโมเดลที่เหมาะสมให้

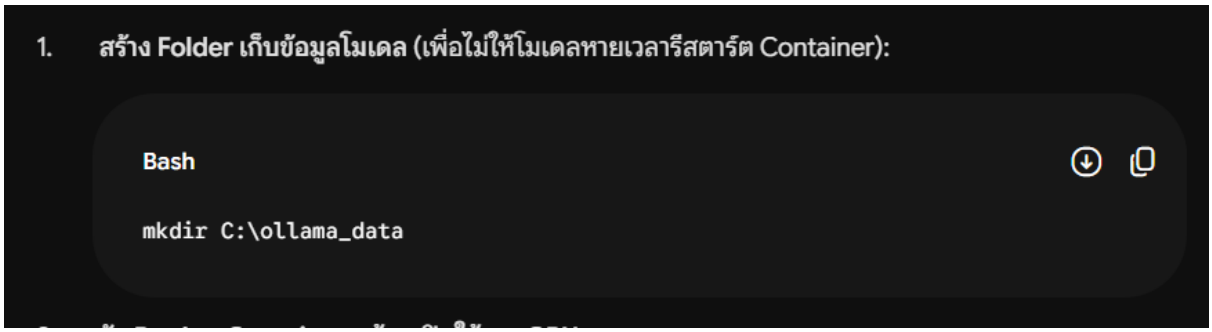
ขั้นตอนการติดตั้งอย่างละเอียด (ตั้งแต่ Docker จนถึง VSCode)



ตามภาพ ติดตั้งdocker และ (ถ้าติดsystem requirmentในDocker ก็ไปแคปให้ai แก้ดูได้)

Step 2: สร้างและรัน Ollama Container บน Docker

เปิด Command Prompt (CMD) หรือ PowerShell แล้วทำตามขั้นตอนต่อไปนี้:

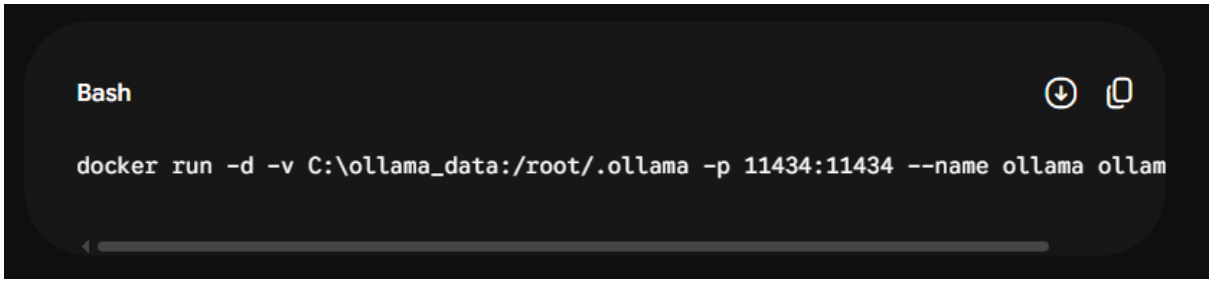


mkdir C:\ollama_data



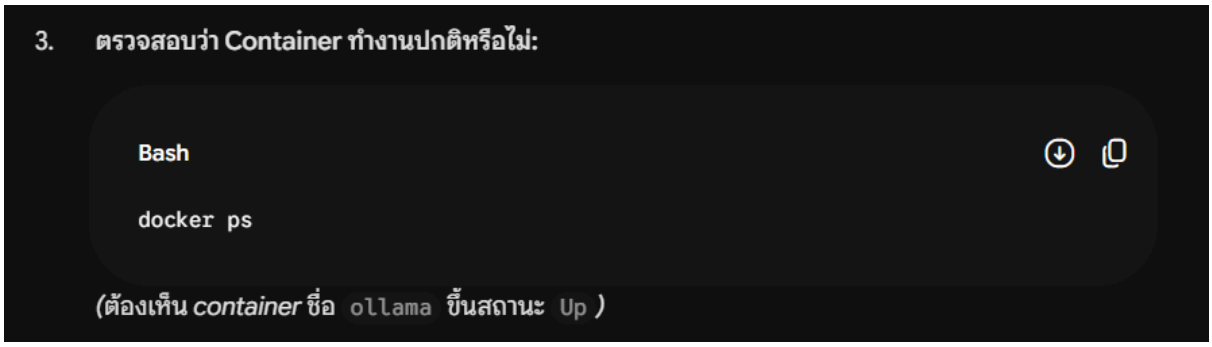
อันนี้จะเป็นการติดตั้งlocal ai และใช้งานผ่านGpu

docker run -d --gpus=all -v C:\ollama_data:/root/.ollama -p 11434:11434 --name ollama ollama/ollama

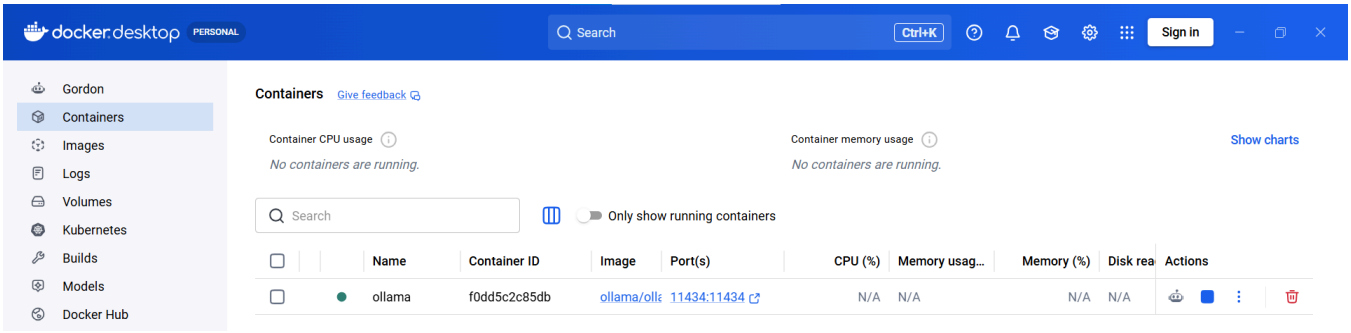


ส่วนอันนี้กรณีคอมไม่มีการ์ดจอ Gpu

docker run -d -v C:\ollama_data:/root/.ollama -p 11434:11434 --name ollama ollama/ollama



ทำตามขั้นที่3แล้วต้องเห็น ollama ใน Docker กำลังรันเหมือนรูปข้างล่าง ↓



Step 3: ดาวน์โหลดโมเดลเข้า Ollama

ใช้คำสั่ง docker exec ส่งคำสั่งเข้าไปดาวน์โหลดโมเดลใน Container:

1. ดาวน์โหลด Qwen 2.5 Coder (1.5B): Database Mart

Bash

```
docker exec -it ollama ollama pull qwen2.5-coder:1.5b
```

2. ดาวน์โหลด Llama 3.2 (3B):

Bash

```
docker exec -it ollama ollama pull llama3.2:3b
```

3. ทดสอบพิมพ์คุยใน Terminal (ตัวอย่าง Qwen):

Bash

```
docker exec -it ollama ollama run qwen2.5-coder:1.5b
```

(ทดสอบพิมพ์ทักทาย หรือสั่งให้เขียนโค้ด แล้วกด `Ctrl + D` เพื่อออกจากหน้าแชท)

*อันนี้ตัวโมเดลจะแตกต่างตามสเปคเครื่องที่ได้บอกและที่หาให้

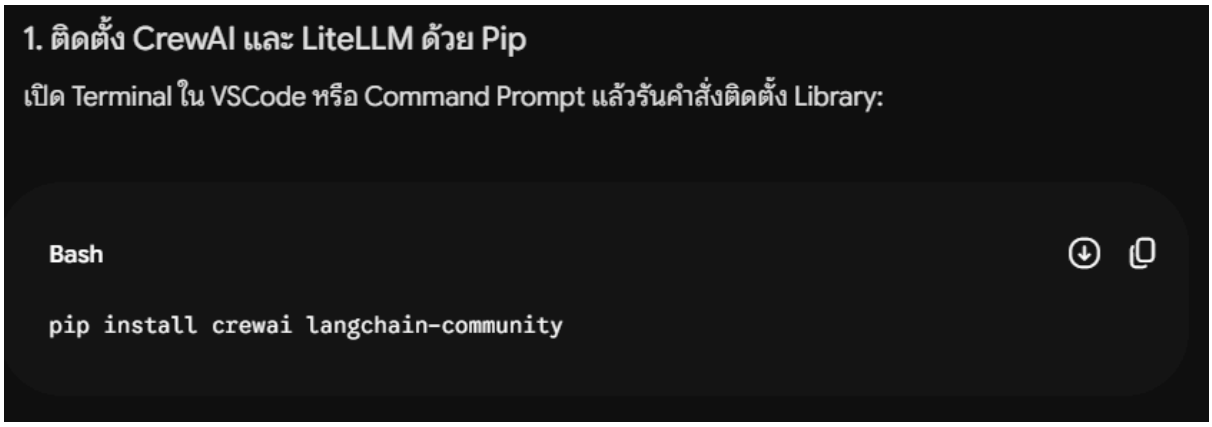
Step 4: เชื่อมต่อใช้งานใน VSCode

เพื่อใช้งาน AI ช่วยเขียนโค้ด (Copilot Style) ภายใน VSCode ให้ทำตามขั้นตอนดังนี้:

ฉันใช้ CrewAI ร่วมกับ Ollama (Docker)

🤖 CrewAI คืออะไร และทำไมถึงเหมาะกับโปรเจกต์นี้?

- **แนวคิด:** แทนที่จะใช้ AI ตัวเดียวตอบคำถามแบบ Chatbot ตัว CrewAI จะให้เราสร้าง **"ทีม AI" (Agents)** ที่มีบทบาทต่างกัน เช่น:
  - **Agent 1 (Researcher):** ค้นหาและรวบรวมข้อมูล
  - **Agent 2 (DevOps Engineer):** เขียน Script Dockerfile หรือ Kubernetes Config
- **การเชื่อมต่อ:** สามารถสั่งให้ CrewAI วิ่งไปดึงโมเดล qwen2.5-coder:1.5b หรือ llama3.2:3b จาก **Ollama บน Docker (http://localhost:11434)** ที่รันเตรียมไว้ได้ทันที



pip install crewai langchain-community

2. เขียน Script Python ทดสอบสั่งงาน CrewAI + Ollama

สร้างไฟล์ชื่อ app.py ในโฟลเดอร์โปรเจกต์ของคุณ แล้วคัดลอกโค้ดด้านล่างนี้ไปวาง:



```python
from crewai import Agent, Task, Crew, Process
from langchain_community.llms import Ollama

# 1. เชื่อมต่อกับ Ollama ที่รันอยู่ใน Docker (ใช้โมเดลที่คุณโหลดไว้)
local_llm = Ollama(
    model="qwen2.5-coder:1.5b",
    base_url="http://localhost:11434"
)

# 2. สร้าง Agent (กำหนดบทบาทให้ AI)
devops_engineer = Agent(
    role='Senior DevOps Engineer',
    goal='ช่วยออกแบบและเขียนสคริปต์การตั้งค่าระบบ Infrastructure',
    backstory='คุณคือผู้เชี่ยวชาญด้าน Docker, CI/CD และ Local AI Infrastructure',
    verbose=True,
    llm=local_llm
)

# 3. กำหนดงาน (Task) ให้ Agent ทำ
task1 = Task(
    description='เขียนไฟล์ docker-compose.yml สำหรับรัน Web App ชนิด Python Flask คู่กับ Post
    expected_output='โค้ด docker-compose.yml ที่สมบูรณ์พร้อมคำอธิบายการตั้งค่า',
    agent=devops_engineer
)

# 4. รันกระบวนการทำงานของ Crew
crew = Crew(
    agents=[devops_engineer],
    tasks=[task1],
    process=Process.sequential
)

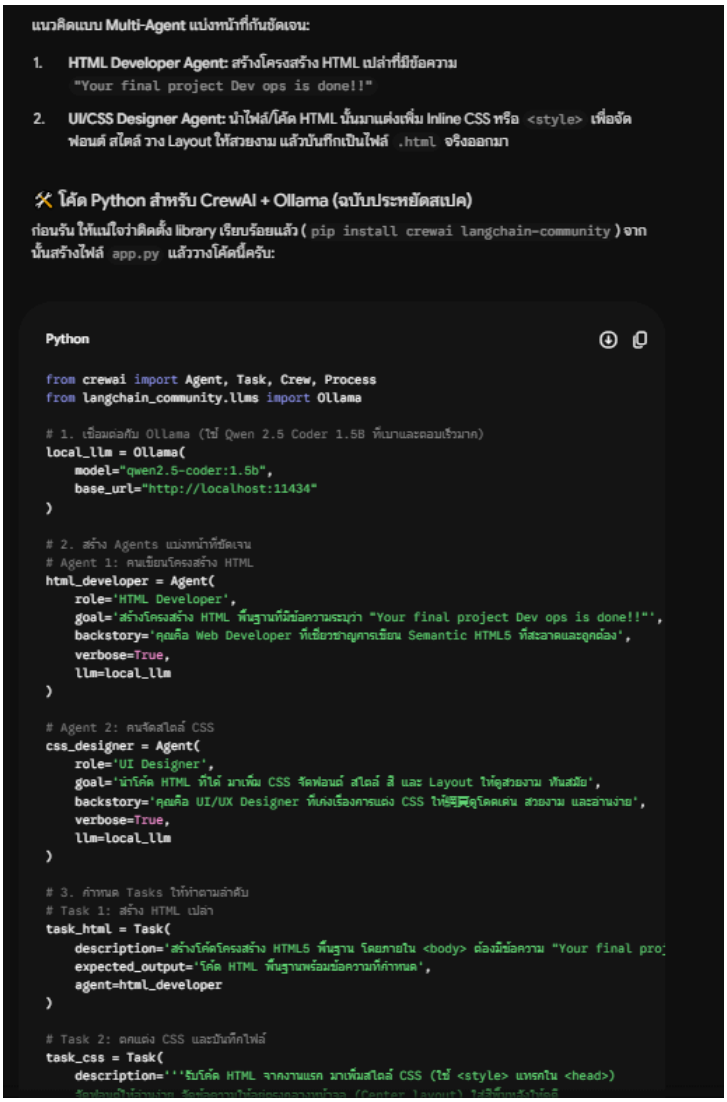
print("### กำลังเริ่มทำงานร่วมกับ CrewAI + Local Ollama... ###")
result = crew.kickoff()

print("\n\n########################")
print("## ผลลัพธ์จาก CREW AI ##")
print("########################\n")
print(result)
```

-มันจะให้สคริปที่เอาไว้สั่ง Ai agent ว่าให้ทำอะไรบ้าง จะพร็อมคุยปรับแต่งมาให้ahentมีหน้าที่ทำอะไรก็ได้

ตัวอย่างที่ฉันอยากจะให้agentทำอะไรตามพร้อมด้านล่าง

*มาคุยเรื่องเขียน Script Python ทดสอบสั่งงาน CrewAI + Ollama
ที่นายให้มาฉันว่ามันหนักไปสำหรับจะสั่งโมเดลแล้วรันในสเปคคอมฉันเอาเป็นแบบนี้ไหม ให้มันสร้างไฟล์htmlว่า"Your final project Dev ops is done!!"แล้วก็ให้เอเจนต์แต่ละตัวทำหน้าที่ของมัน ตัวนึงสร้างโครงสร้างhtml อีกตัวจัดฟอนต์ สไตล์ เลเอาต์ อะไรแบบนี้



แนวคิดแบบ Multi-Agent แบ่งหน้าที่กันชัดเจน:

1. HTML Developer Agent: สร้างโครงสร้าง HTML เปล่าที่มีข้อความ "Your final project Dev ops is done!!"
2. UI/CSS Designer Agent: นำไฟล์/โค้ด HTML นั้นมาแต่งเพิ่ม Inline CSS หรือ <style> เพื่อจัดฟอนต์ สไตล์ วาง Layout ให้สวยงาม แล้วบันทึกเป็นไฟล์ .html จริงออกมา

🛠️ โค้ด Python สำหรับ CrewAI + Ollama (ฉบับประหยัดสเปค)

ก่อนรัน ให้แน่ใจว่าติดตั้ง library เรียบร้อยแล้ว (pip install crewai langchain-community) จากนั้นสร้างไฟล์ app.py แล้ววางโค้ดนี้ครับ:

Python

```python
from crewai import Agent, Task, Crew, Process
from langchain_community.llms import Ollama

# 1. เชื่อมต่อกับ Ollama (ใช้ Qwen 2.5 Coder 1.5B ที่เบาและตอบเร็วมาก)
local_llm = Ollama(
    model="qwen2.5-coder:1.5b",
    base_url="http://localhost:11434"
)

# 2. สร้าง Agents แบ่งหน้าที่ชัดเจน
# Agent 1: คนเขียนโครงสร้าง HTML
html_developer = Agent(
    role='HTML Developer',
    goal='สร้างโครงสร้าง HTML พื้นฐานที่มีข้อความระบุว่า "Your final project Dev ops is done!!"',
    backstory='คุณคือ Web Developer ที่เชี่ยวชาญการเขียน Semantic HTML5 ที่สะอาดและถูกต้อง',
    verbose=True,
    llm=local_llm
)

# Agent 2: คนจัดสไตล์ CSS
css_designer = Agent(
    role='UI Designer',
    goal='นำโค้ด HTML ที่ได้ มาเพิ่ม CSS จัดฟอนต์ สไตล์ สี และ Layout ให้ดูสวยงาม ทันสมัย',
    backstory='คุณคือ UI/UX Designer ที่เก่งเรื่องการแต่ง CSS ให้หน้าตาโค้ดเดิม สวยงาม และอ่านง่าย',
    verbose=True,
    llm=local_llm
)

# 3. กำหนด Tasks ให้ทำตามลำดับ
# Task 1: สร้าง HTML เปล่า
task_html = Task(
    description='สร้างโค้ดโครงสร้าง HTML5 พื้นฐาน โดยภายใน <body> ต้องมีข้อความ "Your final pro
    expected_output='โค้ด HTML พื้นฐานพร้อมข้อความที่กำหนด',
    agent=html_developer
)

# Task 2: ตกแต่ง CSS และบันทึกไฟล์
task_css = Task(
    description='''รับโค้ด HTML จากงานแรก มาเพิ่มสไตล์ CSS (ใช้ <style> แทรกใน <head>)
```

ก็จะให้โค้ดที่สั่งagentมา

🏃 วิธีการรันและดูผลลัพธ์

1. ตรวจสอบว่า Docker Container ของ Ollama ยังรันอยู่ (`docker ps`)[cite: 1]
2. รันสคริปต์ใน Terminal:

```
Bash

python app.py
```

python app.py

3. ใน Terminal คุณจะเห็น Agent ตัวที่ 1 คุยตอบกับ Ollama เพื่อสร้างโครงสร้าง HTML จากนั้น Agent ตัวที่ 2 จะรับช่วงต่อเพื่อใส่ CSS
4. เมื่อรันเสร็จ จะมีไฟล์ `index.html` เด้งขึ้นมาในโฟลเดอร์โปรเจกต์ของคุณ ดับเบิลคลิกเปิดด้วย Chrome หรือ Edge เพื่อดูหน้าเว็บที่ AI ทั้งสองตัวช่วยกันทำได้ทันทีครับ!

หมายเหตุ Libraryที่โหลดมาของCrew ai อาจจะเกิดerror เพราะเวอร์ชั่นคนละตัวกับLLM

สาเหตุคือ Agent ของ CrewAI เวอร์ชันที่ติดตั้งอยู่ต้องการ LLM ที่เป็นชนิดของ CrewAI แต่ตัวเดิมเป็นชนิดของ LangChain จึงเกิด Pydantic ValidationError ตอนสร้าง Agent

ติดปัญหาอะไร ส่งให้Copilot ในVscode แก้ให้ มันทำได้